

Optymalizacja Kodu na Różne Architektury

Zadanie 2 - Eliminacja Gaussa

Cel zadania

Celem zadania jest implementacja i optymalizacja algorytmu eliminacji Gaussa, z uwzględnieniem mikroarchitektury posiadanego procesora oraz praktycznego wykorzystania technik niskopoziomowej optymalizacji kodu.

Zakres prac:

- Zaimplementuj algorytm eliminacji Gaussa (z lub bez pivotingu).
- Zoptymalizuj implementację pod kątem architektury CPU (cache, SIMD, itp.).
- Zweryfikuj poprawność: wynik optymalizowanej wersji musi odpowiadać wersji referencyjnej.
- Oszacuj liczby operacji (FLOP) i przelicz GFLOPS.
- Przeprowadź pomiary wydajności (opcjonalnie z użyciem PAPI lub równoważnego narzędzia).

Wymagania techniczne:

- Wektoryzacja obliczeń z użyciem instrukcji SIMD (np. AVX, SSE).
- Dbłość o lokalność danych (wykorzystanie pamięci cache).
- Analiza architektury CPU i dostosowanie kodu do jednostek wykonawczych.
- Pomiary czasu wykonania oraz obliczonych GFLOPS-ów.
- Wykonanie testów poprawności dla różnych rozmiarów macierzy.

Dane techniczne

System Operacyjny

```
1 $ hostnamectl
2 Virtualization: wsl
3 Operating System: Ubuntu 24.04.2 LTS
4 Kernel: Linux 6.6.87.1-microsoft-standard-WSL2
5 Architecture: x86-64
```

Kompilator

```
1 $ gcc --version
2 gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
```

Procesor

```
1 $ lscpu
2 Architecture:          x86_64
3   CPU op-mode(s):      32-bit, 64-bit
4   Address sizes:        48 bits physical, 48 bits virtual
5   Byte Order:           Little Endian
6   CPU(s):               24
7   On-line CPU(s) list: 0-23
8   Vendor ID:            AuthenticAMD
9   Model name:           AMD Ryzen 9 7900X 12-Core Processor
10  CPU family:            25
11  Model:                 97
12  Thread(s) per core:    2
13  Core(s) per socket:    12
14  Socket(s):             1
15  Stepping:              2
16  BogoMIPS:              9399.78
17  Flags:                 fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36
18                        clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp
19                        lm constant_tsc rep_good nopl tsc_reliable nonsto p_tsc cpuid extd_apicid
20                        pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 movbe popcnt aes xsave avx f16c
21                        rdrand hypervisor lahf_lm cmp_legacy svm cr8_legacy abm sse4a misalignsse
22                        3dnowprefetch osvw topoext perfctr_core ssbd ibrs ibpb stibp vmmcall
23                        fsgsbase bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap
24                        avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt
25                        xsavec xgetbv1 xsaves avx512_bf16 clzero xsaveerptr arat npt nrip_save
26                        tsc_scale vmcb_clean flushbyasid decodeassists pausefilter pfthreshold
27                        v_vmsave_vmload avx512vbmi umip avx512_vbmi2 gf ni vaes vpclmulqdq
28                        avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid fsrm
29  Caches (sum of all):
30    L1d:                 384 KiB (12 instances)
31    L1i:                 384 KiB (12 instances)
32    L2:                  12 MiB (12 instances)
33    L3:                  32 MiB (1 instance)
```

Weryfikacja poprawności

Układy testowych danych zostały wygenerowane przy pomocy funkcji

```
1 void generate_solvable_system(double **A, double *b, double *true_x,
2                               const int SIZE) {
3     for (int i = 0; i < SIZE; i++) {
4         true_x[i] = ((double)rand() / RAND_MAX) * 20.0 - 10.0;
5     }
6
7     for (int i = 0; i < SIZE; i++) {
8         for (int j = 0; j < SIZE; j++) {
9             A[i][j] = ((double)rand() / RAND_MAX) * 20.0 - 10.0;
10        }
11    }
12
13    for (int i = 0; i < SIZE; i++) {
14        b[i] = 0.0;
15        for (int j = 0; j < SIZE; j++) {
16            b[i] += A[i][j] * true_x[j];
17        }
18    }
19 }
```

Sprawdzenie rozwiązania wykonano przy pomocy funkcji

```
1 int verify_solution(double *x, double *true_x, const int SIZE) {
2     double tolerance = 1e-4;
3
4     for (int i = 0; i < SIZE; i++) {
5         if (fabs(x[i] - true_x[i]) > tolerance) return 1; // Verification failed
6     }
7     return 0; // Verification successful
8 }
```

Zaimplementowana wersja algorytmu **nie zawierała pivotingu**, a testy były przeprowadzane na macierzach kwadratowych bez wielowątkowości.

Obliczenie GFLOPS-ów

Do obliczenia GFLOPS-ów wykorzystano narzędzie perf. Przykładowy pomiar

```
1 $ gcc -mfma -O2 ge3.c -o ge3
2 $ perf stat -e fp_ret_sse_avx_ops.all,task-clock ./ge3 3333
3 Time (seconds): 1.851049e+00
4
5 Performance counter stats for './ge3 3333':
6
7      24767274143      fp_ret_sse_avx_ops.all:u      #    13.920 G/sec
8      1779.26 msec task-clock:u      #    0.908 CPUs utilized
9
10     1.958770318 seconds time elapsed
11
12     1.729572000 seconds user
13     0.047236000 seconds sys
```

Wersja podstawowa

Algorytm eliminacji Gaussa bez optymalizacji.

```
1 int gaussian_elimination(double **A, double *b, double *x, const int SIZE) {
2     // Forward elimination
3     for (int k = 0; k < SIZE - 1; k++) {
4         // Eliminate column k in rows below
5         for (int i = k + 1; i < SIZE; i++) {
6             double factor = A[i][k] / A[k][k];
7             for (int j = k; j < SIZE; j++) {
8                 A[i][j] -= factor * A[k][j];
9             }
10            b[i] -= factor * b[k];
11        }
12    }
13
14    // Back substitution
15    for (int i = SIZE - 1; i >= 0; i--) {
16        x[i] = b[i];
17        for (int j = i + 1; j < SIZE; j++) {
18            x[i] -= A[i][j] * x[j];
19        }
20        x[i] /= A[i][i];
21    }
22
23    return 0; // Success
24 }
```

Macierz (NxN)	Czas algorytmu (s)	Całkowity czas (s)	Całkowite GFLOPS
333	2.709000e-03	4.723702e-03	5.956
1111	1.090970e-01	1.210884e-01	7.830
2222	1.215120e+00	1.265453e+00	5.853
3333	6.197356e+00	6.328828e+00	4.265
6666	4.859530e+01	5.015407e+01	4.009

Table 1: Wyniki pomiarów (perf) dla wersji podstawowej

Wersja z wektoryzacją

Algorytm eliminacji Gaussa z wektoryzacją przy użyciu instrukcji AVX2.

```
1 void kernel_eliminate_8(double *A_pivot, double *A_target, int jj,
2                        double factor, const int SIZE) {
3     register __m256d factor_vec;
4     register __m256d pivot_vec_1, pivot_vec_2;
5     register __m256d target_vec_1, target_vec_2;
6
7     factor_vec = _mm256_set1_pd(factor);
8
9     pivot_vec_1 = _mm256_loadu_pd(&A_pivot[jj]);
10    pivot_vec_2 = _mm256_loadu_pd(&A_pivot[jj + 4]);
11
12    target_vec_1 = _mm256_loadu_pd(&A_target[jj]);
13    target_vec_2 = _mm256_loadu_pd(&A_target[jj + 4]);
14
15    target_vec_1 = _mm256_fmadd_pd(factor_vec, pivot_vec_1, target_vec_1);
16    target_vec_2 = _mm256_fmadd_pd(factor_vec, pivot_vec_2, target_vec_2);
17
18    _mm256_storeu_pd(&A_target[jj], target_vec_1);
19    _mm256_storeu_pd(&A_target[jj + 4], target_vec_2);
20 }
21
22 int gaussian_elimination(double **A, double *b, double *x, const int SIZE) {
23     double factor;
24     int i, j, k, jj;
25
26     // Forward elimination
27     for (k = 0; k < SIZE - 1; k++) {
28         for (i = k + 1; i < SIZE; i++) {
29             factor = A[i][k] / A[k][k];
30             for (jj = k; jj <= SIZE - 8; jj += 8) {
31                 kernel_eliminate_8(A[k], A[i], jj, factor, SIZE);
32             }
33             for (j = jj; j < SIZE; j++) A[i][j] -= factor * A[k][j];
34             b[i] -= factor * b[k];
35         }
36     }
37
38     // Back substitution
39     for (int i = SIZE - 1; i >= 0; i--) {
40         x[i] = b[i];
41         for (int j = i + 1; j < SIZE; j++) {
42             x[i] -= A[i][j] * x[j];
43         }
44         x[i] /= A[i][i];
45     }
46
47     return 0; // Success
48 }
```

Macierz (NxN)	Czas algorytmu (s)	Całkowity czas (s)	Całkowite GFLOPS
333	1.629000e-03	3.580134e-03	8.041
1111	6.874200e-02	8.100482e-02	11.642
2222	8.545080e-01	9.089234e-01	8.186
3333	4.543042e+00	4.670241e+00	5.718
6666	3.929290e+01	4.025762e+01	4.989

Table 2: Wyniki pomiarów (perf) dla wersji z wektoryzacją

Wersja z wektoryzacją i blokowaniem

Algorytm eliminacji Gaussa z wektoryzacją przy użyciu instrukcji AVX2 oraz efektywniejszym wykorzystaniem pamięci cache poprzez blokowanie.

```
1  #define BLOCK_SIZE 32
2
3  void kernel_eliminate_8(double *A_pivot, double *A_target, int jj,
4                        double factor, const int SIZE) {
5      register __m256d factor_vec;
6      register __m256d pivot_vec_1, pivot_vec_2;
7      register __m256d target_vec_1, target_vec_2;
8
9      factor_vec = _mm256_set1_pd(factor);
10
11     pivot_vec_1 = _mm256_loadu_pd(&A_pivot[jj]);
12     pivot_vec_2 = _mm256_loadu_pd(&A_pivot[jj + 4]);
13
14     target_vec_1 = _mm256_loadu_pd(&A_target[jj]);
15     target_vec_2 = _mm256_loadu_pd(&A_target[jj + 4]);
16
17     target_vec_1 = _mm256_fmadd_pd(factor_vec, pivot_vec_1, target_vec_1);
18     target_vec_2 = _mm256_fmadd_pd(factor_vec, pivot_vec_2, target_vec_2);
19
20     _mm256_storeu_pd(&A_target[jj], target_vec_1);
21     _mm256_storeu_pd(&A_target[jj + 4], target_vec_2);
22 }
23
24 int gaussian_elimination(double **A, double *b, double *x, const int SIZE) {
25     double factor;
26     int i, j, k, jj, kk, kk_end;
27
28     // Forward elimination
29     for (kk = 0; kk < SIZE - 1; kk += BLOCK_SIZE) {
30         kk_end = (kk + BLOCK_SIZE < SIZE) ? kk + BLOCK_SIZE : SIZE;
31
32         // Process current block
33         for (k = kk; k < kk_end - 1; k++) {
34             for (i = k + 1; i < kk_end; i++) {
35                 factor = A[i][k] / A[k][k];
36                 for (jj = k; jj <= SIZE - 8; jj += 8) {
37                     kernel_eliminate_8(A[k], A[i], jj, factor, SIZE);
38                 }
39                 for (j = jj; j < SIZE; j++) A[i][j] -= factor * A[k][j];
40                 b[i] -= factor * b[k];
41             }
42         }
43
44         // Update remaining rows using the processed block
45         for (i = kk_end; i < SIZE; i++) {
46             for (k = kk; k < kk_end; k++) {
47                 factor = A[i][k] / A[k][k];
48                 for (jj = k; jj <= SIZE - 8; jj += 8) {
49                     kernel_eliminate_8(A[k], A[i], jj, factor, SIZE);
50                 }
51                 for (j = jj; j < SIZE; j++) A[i][j] -= factor * A[k][j];
52                 b[i] -= factor * b[k];
53             }
54         }
55     }
```

```

54     }
55 }
56
57 // Back substitution
58 for (int i = SIZE - 1; i >= 0; i--) {
59     x[i] = b[i];
60     for (int j = i + 1; j < SIZE; j++) {
61         x[i] -= A[i][j] * x[j];
62     }
63     x[i] /= A[i][i];
64 }
65
66 return 0; // Success
67 }

```

Macierz (NxN)	Czas algorytmu (s)	Całkowity czas (s)	Całkowite GFLOPS
333	1.447000e-03	3.285216e-03	8.875
1111	5.827000e-02	7.104668e-02	13.227
2222	4.786020e-01	5.327180e-01	13.986
3333	1.851049e+00	1.958770e+00	13.920
6666	1.482420e+01	1.588368e+01	12.712

Table 3: Wyniki pomiarów (perf) dla wersji z wektoryzacją i blokowaniem

Wnioski

Czas oraz liczba operacji w generowaniu macierzy, weryfikacji rozwiązania oraz w samym algorytmie eliminacji podczas podstawienia wstecznego były pomijalnie małe w porównaniu do czasu eliminacji, więc do obliczenia GFLOPS-ów wystarczyło użycie całkowitego czasu wykonania programu.

Prefetchowanie danych mierzalnie nie wpływało na czas wykonania lub nawet pogarszało wyniki dla wersji z wektoryzacją i blokowaniem, więc zostało pominięte.

Całkowita liczba operacji zmiennoprzecinkowych przybliżona jest do teoretycznej złożoności $(2/3)N^3$, przy macierzy $N \times N$.

Zastosowanie optymalizacji znacznie poprawiło wyniki algorytmu. Poprawa zależy od rozmiaru macierzy, gdzie dla większych macierzy różnice są bardziej zauważalne. Ręczna implementacja wektoryzacji przyniosła poprawę o około 25% przy rozmiarze macierzy 3333, co może wynikać z faktu, że kompilator tylko w pewnym stopniu wykorzystał automatyczną wektoryzację. Wersja z blokowaniem natomiast przyniosła dużą poprawę o około 50% przy rozmiarze macierzy 3333, więc widzimy, że w tym przypadku efektywne ładowanie danych do pamięci cache ma kluczowe znaczenie.